



Gesture Controlled Android Game

Presented by Team Dominator069

Members: Rishabh , Navneet , Shardul , Rakshit

Introduction & Objectives

We are developing an AI-powered, gesture-controlled racing game for Android using the Qualcomm Innovation Development Kit (QIDK).

AI-Powered Control

Replaces traditional controls with hand gesture recognition for an interactive experience.

Key Objectives

- Collect & preprocess gesture data.
- Train & fine-tune a recognition model.
- Integrate into an Android racing game.
- Map gestures to game controls.



Data Collection & Model Training

Data Collection: Done

- Developed pipeline for capturing gesture samples.
- MediaPipe extracts hand landmarks (input features).
- Collected 200 samples per gesture across 8 gestures, stored in gestures.csv.
- Dataset split: 70% training, 30% test.

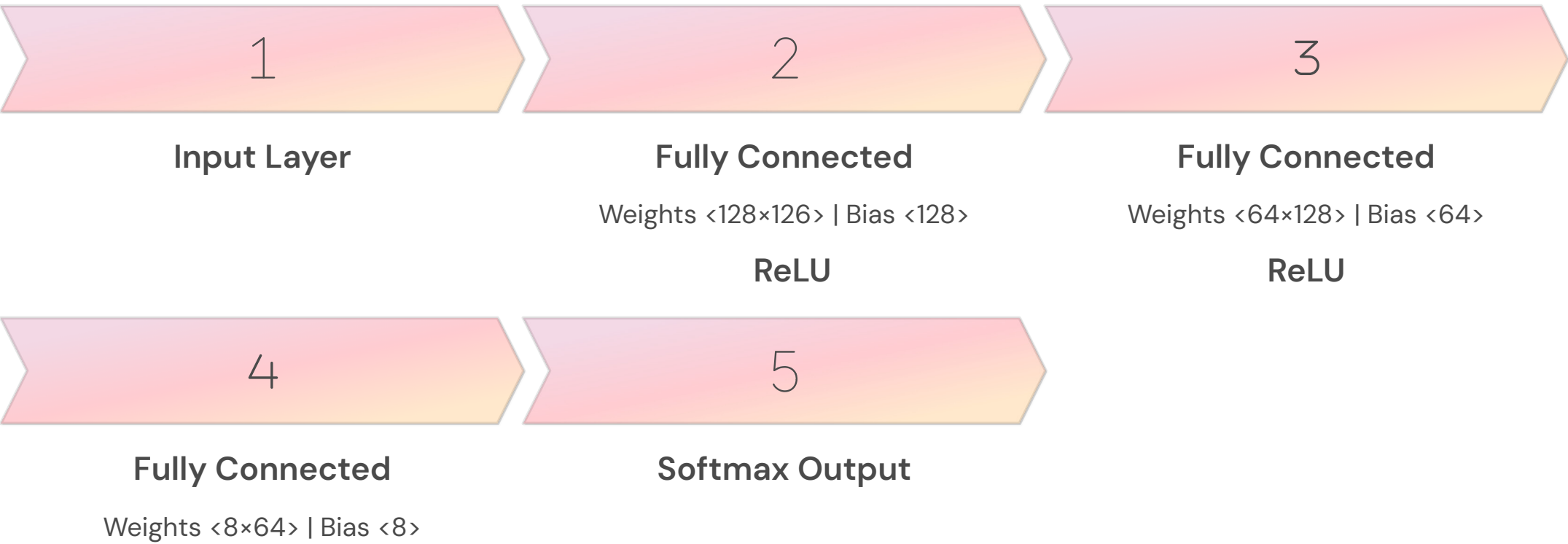
Model Training: Done

- Trained TensorFlow classification model.
- Converted to TensorFlow Lite (TFLite) for mobile.
- Confirmed TFLite model runs correctly on PC.
- Computed confusion matrix and loss curve.
- Achieved solid baseline accuracy distinguishing gestures.



Model Architecture

Our sequential model architecture processes hand landmark data through multiple layers for gesture classification.



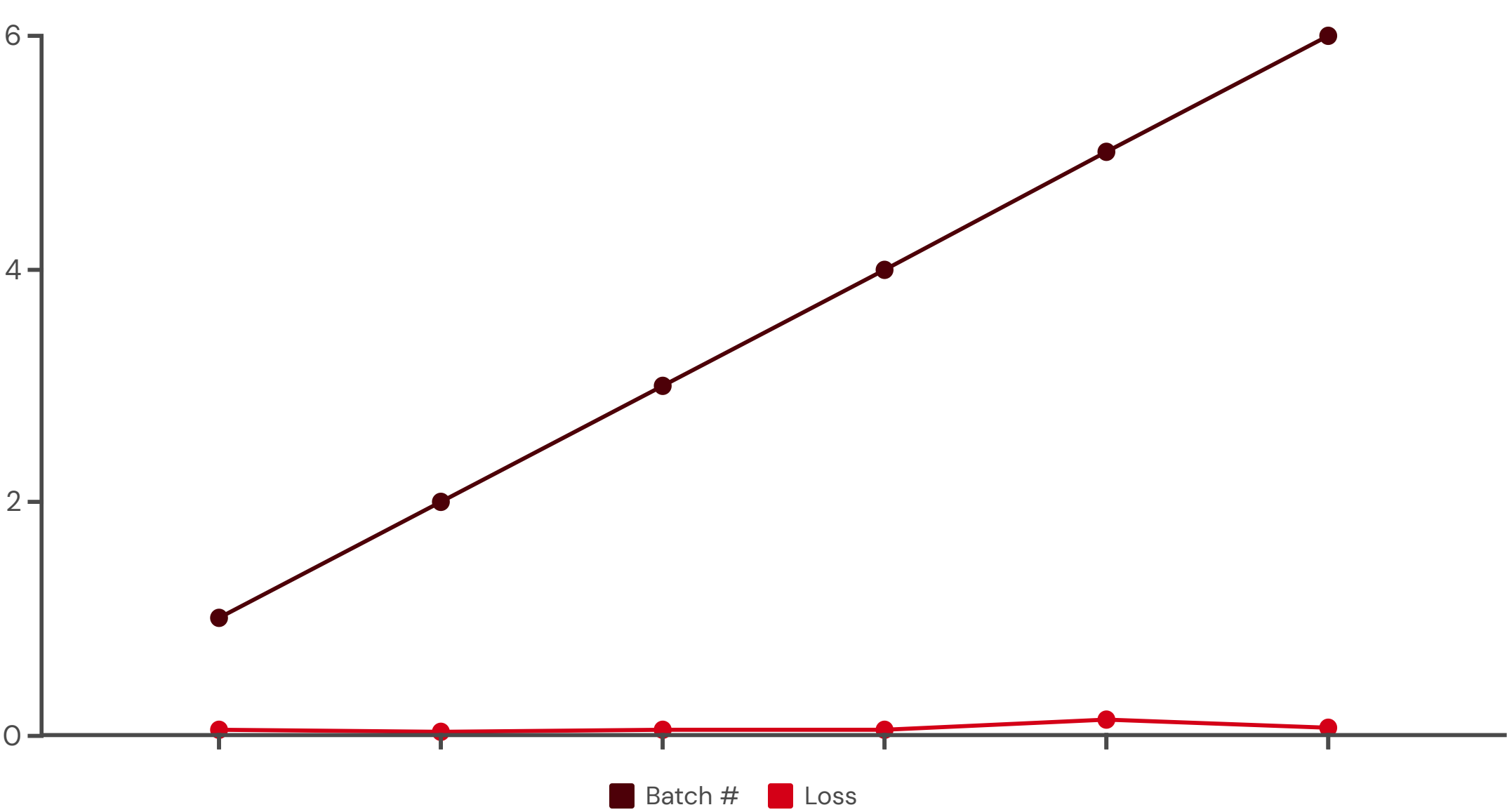
Model Evaluation: Confusion Matrix

The confusion matrix illustrates our model's ability to correctly classify different hand gestures.

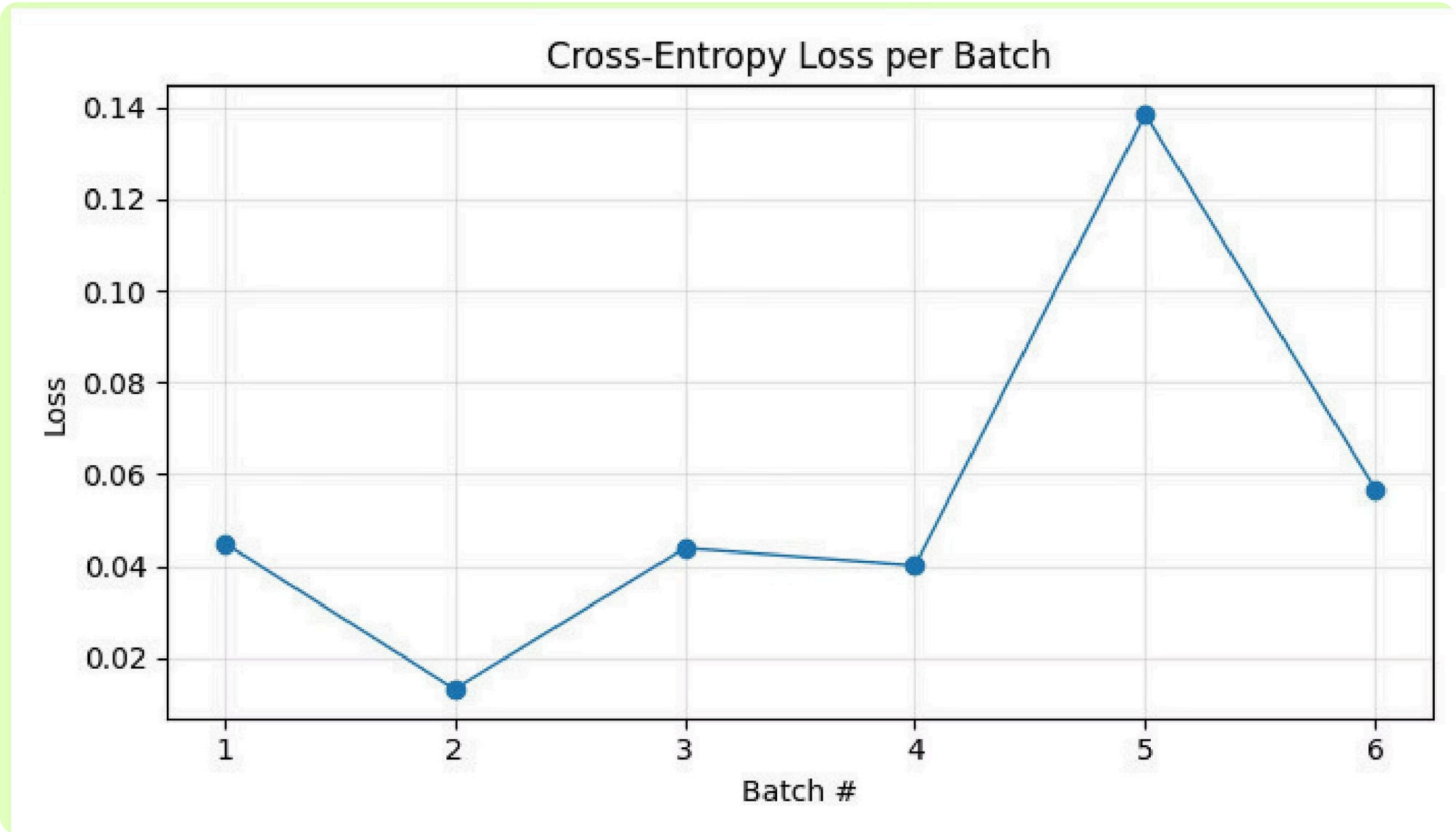
Acc+Left	87	0	0	0	2	0	0	0
Acc+Right	0	63	0	0	0	0	0	0
Accelerate	0	0	52	0	0	0	0	0
Left	0	0	0	86	6	0	0	0
Right	0	0	0	0	128	0	0	0
Stop	0	0	0	0	0	58	0	0
Stop+Left	0	0	0	1	0	0	84	0
Stop+Right	0	0	0	0	1	0	0	96

Loss Curve & Classification Report

Cross-Entropy Loss per Batch



Loss decreases, then peaks at Batch 5 before decreasing again.



Classification Report

Metric	Precision	Recall	F1-Score	Support
Acc+Left	1	0.98	0.99	89
Acc+Right	1	1	1	63
Accelerate	1	1	1	52
Left	0.99	0.93	0.96	92
Right	0.93	1	0.97	128
Stop	1	1	1	58
Stop+Left	1	0.99	0.99	85
Stop+Right	1	0.99	0.99	97
Accuracy	0.98			664
Macro Avg	0.99	0.99	0.99	664
Weighted Avg	0.99	0.98	0.98	664

High precision, recall, and F1-scores across all gestures, with an overall accuracy of 98%.

Quantization & Benchmarking

Model Conversion And Testing

Converted to DLC format and quantized to int8 and w8a16 data types.

Tested the model in 5 modes using snpe-net-run.

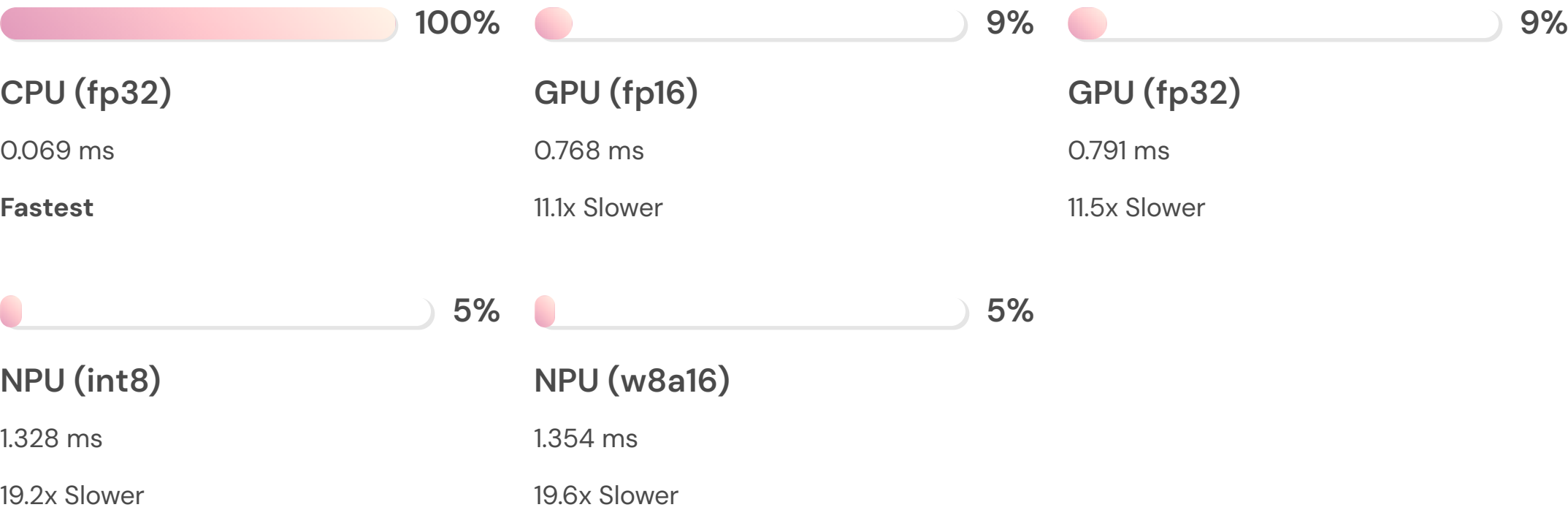
Accuracy Maintained

99.54% accuracy across all tests, confirming zero accuracy loss after quantization.

CPU Performance

CPU was unexpectedly 10x faster due to the model's lightweight nature.

Benchmarking Report: Latency Comparison



All backends maintained **99.54% accuracy** after quantization.

Android App Workflow & Functionalities

Workflow



Camera Feed

Capture camera feed



Frame Preprocessing

Prepare frames for analysis



MediaPipe Hand Detection

Identify hand landmarks



Gesture Classification

Classify gestures using TFLite model



Game Control Mapping

Map gestures to game controls



Game Response

Game responds to detected gestures

Functionalities

Camera Control

Manage camera permissions and select between front or back cameras for optimal input.

Live Preview

Display real-time camera feed with an option to show processing overlays for visual feedback.

Processing Pipeline

An internal, efficient pipeline handles frame capture, pre-processing, and gesture mapping.

Backend Configuration

Customize the processing unit for the gesture model, choosing between CPU, GPU, or NPU.

Debug Tools

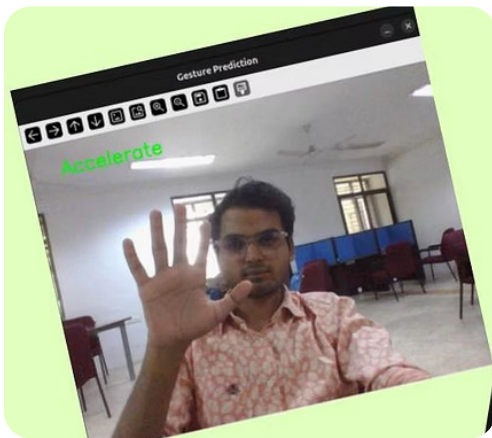
Toggle landmark visualization and access comprehensive debugging information for development.

Low-Latency Loop

Ensure responsive and fluid gameplay through an optimized, low-latency control loop.

Model Flexibility

Seamlessly switch between different gesture recognition models to adapt to various use cases.



Qualcomm NPU Integration & Benchmarking Analysis

NPU Integration

- MediaPipe Hands successfully ran on Qualcomm NPU.
- Pre/post-processing rewritten from Python to Kotlin.
- App supports CPU, GPU, NPU (MediaPipe API), NPU (Qualcomm QNN API).
- NPU execution verified via logcat.
- Current limitation: accuracy issues (tuning in progress).
- Single-hand detection (multi-hand support coming).

Benchmarking Insights

- MediaPipe framework is the primary bottleneck in the full pipeline.
- GPU delegation provides significant speedup on compatible hardware.
- High-end GPUs may perform poorly due to overhead/incompatibility.
- Lightweight classifier model is faster on CPU than NNAPI on some mobile devices.
- Performance bottleneck is heavier MediaPipe components, not the classifier.



Evaluation Metrics & Methodology

Our comprehensive evaluation framework ensures robust assessment of model performance and real-world applicability.

Key Performance Metrics:

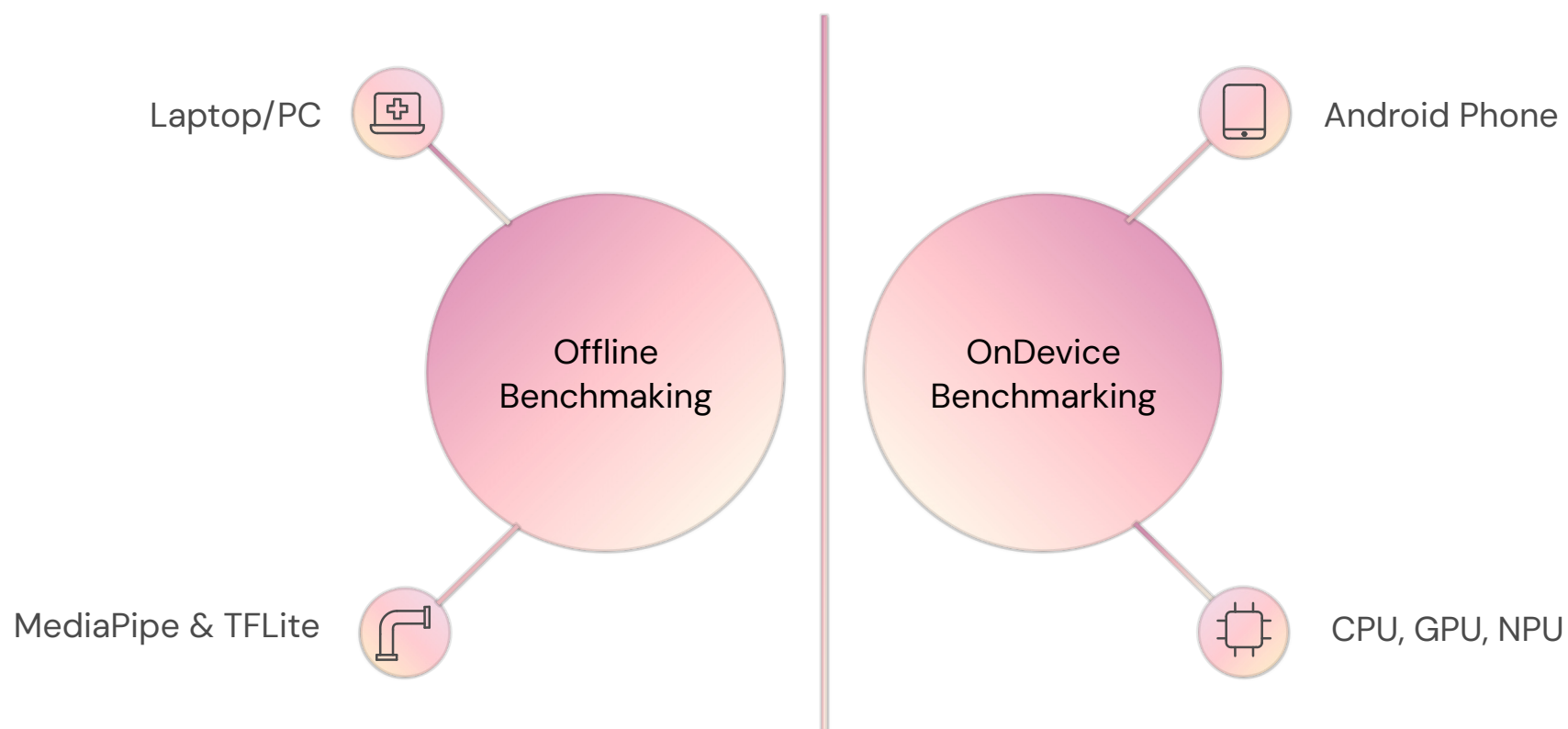
Inference Time Analysis

- Mean inference time (ms)
- Median inference time (ms)
- Standard deviation
- Min / Max latency

Performance Indicators

- 95th percentile (P95)
- 99th percentile (P99)
- Estimated FPS = $1000 / \text{mean_time}$

Evaluation Methodology:



Offline Benchmarking

Python-based benchmarking on laptop/PC for full pipeline: MediaPipe Hand Detection + Landmark Extraction + TFLite/Keras Gesture Model.

On-device Benchmarking

Android benchmarking tool measures pure model inference on CPU / GPU / NPU using NNAPI or GPU delegate, including complete pipeline on phones.

Real-world Performance

All evaluation pipelines designed to reflect real-world performance in the actual app and in QIDK deployment.

Benchmarking Analysis: Laptops & Servers

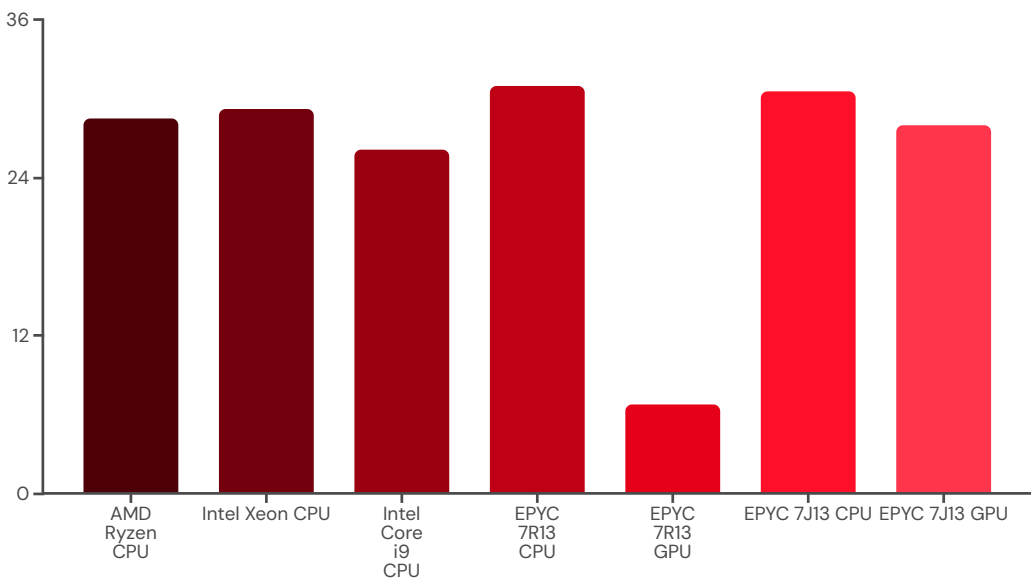
This table provides a granular view of inference metrics for various devices and delegation strategies, offering the raw data behind our analysis.

Device	Delegate	Mean (ms)	Median (ms)	P95 (ms)	FPS
AMD Ryzen 5950X	CPU	28.5	27.1	32.0	35.1
Intel Xeon 8370C	CPU	29.2	28.0	33.5	34.2
Intel Core i9-11900K	CPU	26.1	25.5	30.2	38.3
EPYC 7R13	CPU	31.0	30.0	36.0	32.3
EPYC 7R13	GPU (Tesla T4)	6.8	6.5	7.5	147.1
EPYC 7J13	CPU	30.5	29.8	35.0	32.8
EPYC 7J13	GPU	28.0	27.0	31.5	35.7

Visual Performance Comparison

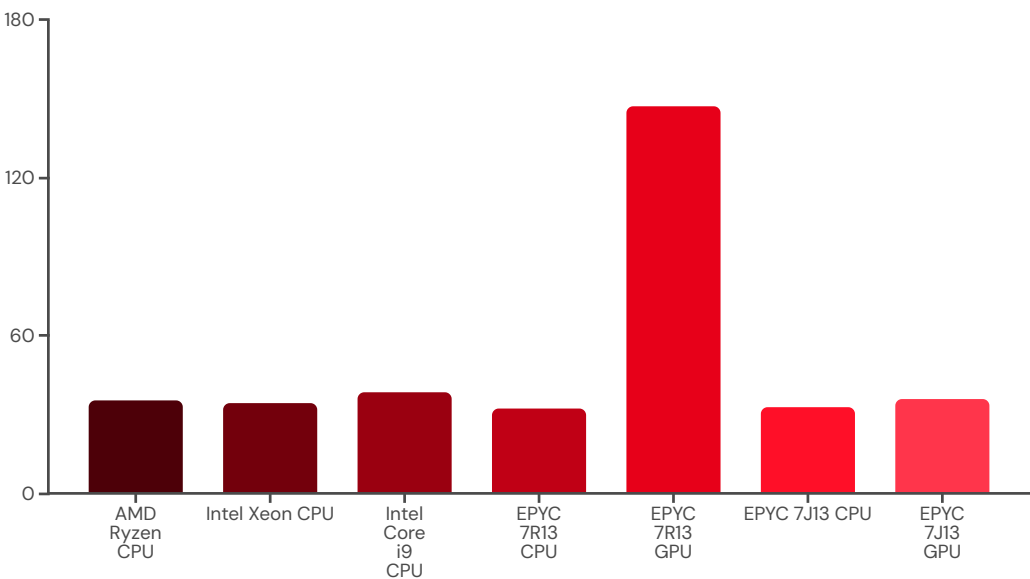
Latency Across Devices (Mean ms)

Lower is better.



FPS Performance Ranking

Higher is better.



MediaPipe Framework Bottleneck

Full pipeline latency (25–32ms) significantly higher than classifier-only (<1ms), identifying MediaPipe as the primary bottleneck.



GPU Provides Significant Speedup

EPYC 7R13 servers with Tesla T4 GPU demonstrate substantial speedup, achieving a mean of 6.8ms (~147 FPS) compared to CPU-only.



Power ≠ Performance

High-end GPU EPYC 7J13 exhibits unexpectedly poor performance (around 28ms) due to initialization overhead and incompatibility issues.



Critical GPU Selection

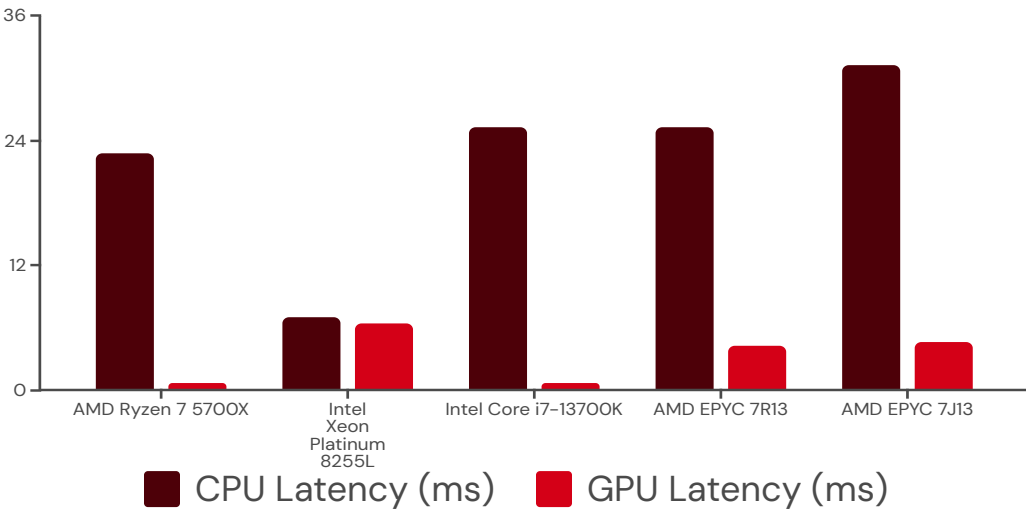
Optimal GPU selection must prioritize compatibility and efficiency over raw computational specifications alone to avoid performance regressions.

Per-Device Performance Breakdown: PC/Server Processors

Detailed Analysis of CPU vs GPU Delegate Performance

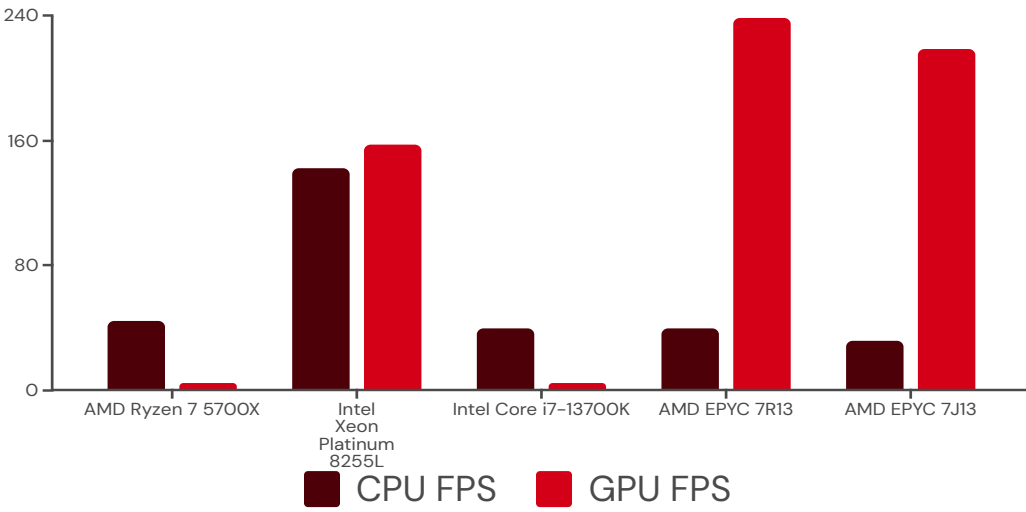
CPU vs GPU Latency (ms)

Lower is better. This chart compares the mean inference time for CPU-only execution against GPU delegate execution on the same processor.



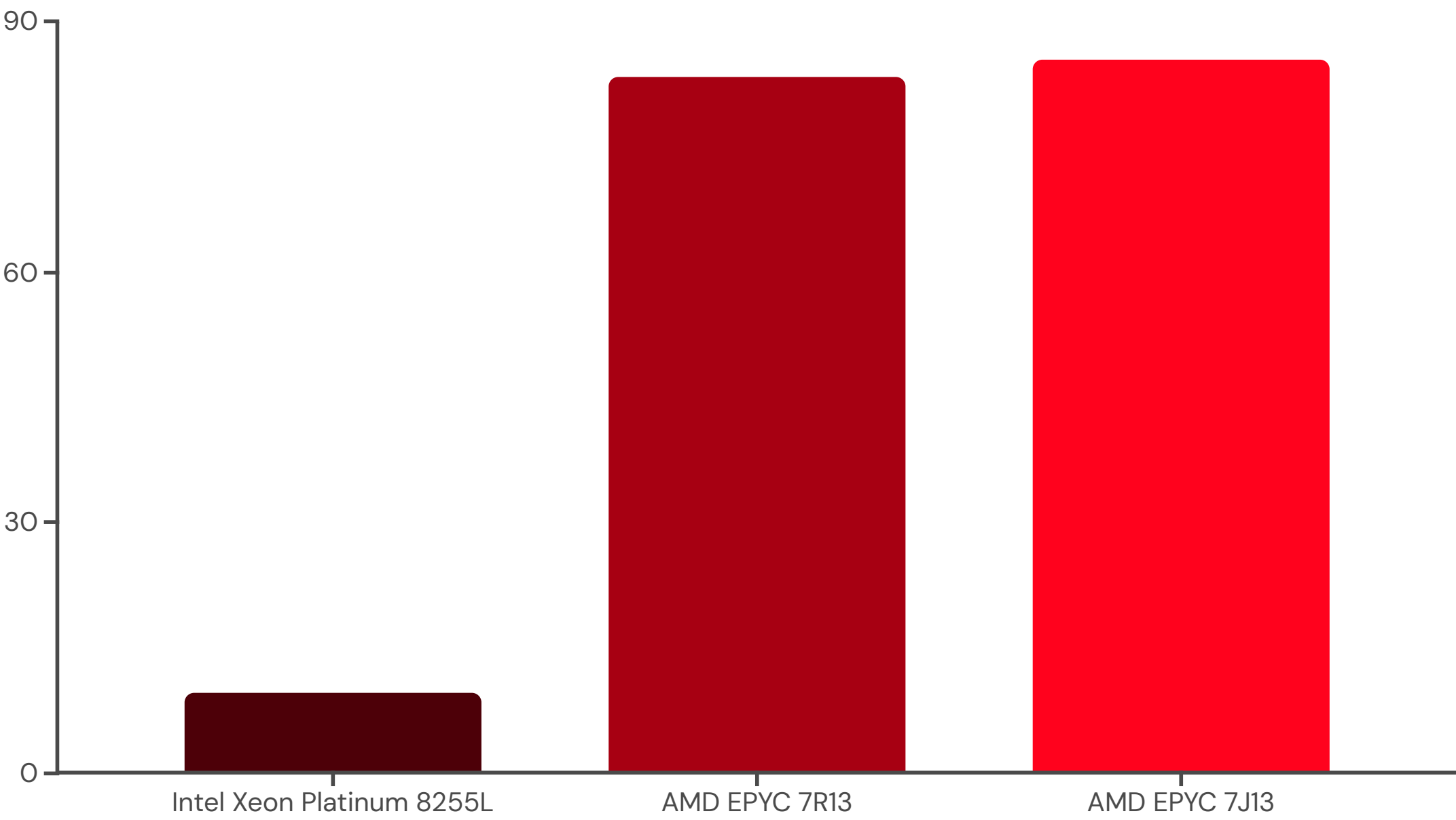
CPU vs GPU FPS

Higher is better. This chart illustrates the Frames Per Second achieved by CPU-only versus GPU delegate processing on various server and PC configurations.



Performance Improvement with GPU Delegate

Percentage improvement in latency when utilizing GPU acceleration compared to CPU-only execution for compatible hardware.



Key Findings

- ➔ **Significant GPU Speedup**
GPU provides a **2-6x speedup** over CPU on compatible hardware, dramatically reducing inference latency.
- ➔ **EPYC 7R13 – Optimal Acceleration**
The AMD EPYC 7R13 processor shows **exceptional GPU acceleration** with a 6x speedup, making it highly efficient.
- ➔ **EPYC 7J13 – Unexpected Performance**
Despite being a high-end GPU, the AMD EPYC 7J13 exhibits **sub-optimal GPU performance** likely due to overhead or incompatibility issues.
- ➔ **Intel Xeon Platinum – Consistent Performer**
The Intel Xeon Platinum 8255L demonstrates the **best overall performance** across both CPU and GPU delegates, offering a balanced solution.
- ➔ **MediaPipe Remains Bottleneck**
The MediaPipe framework continues to be the primary bottleneck in the full pipeline, not the classifier model itself.

Cross-Platform Performance Comparison: Android vs PC/Server

Comprehensive Analysis of Performance Across All Platforms

Performance Tiers: Latency & FPS

A breakdown of inference performance across best, mid-range, and worst-case scenarios for both Android and PC/Server platforms.

Best Case Scenarios

- Android Flagship (Pixel 8 NPU):** 0.009ms, 111,111 FPS
- PC/Server (AMD EPYC 7R13 GPU):** 4.20ms, 238.1 FPS
- PC/Server (Intel Xeon Platinum GPU):** 6.36ms, 157.2 FPS

Mid-Range Scenarios

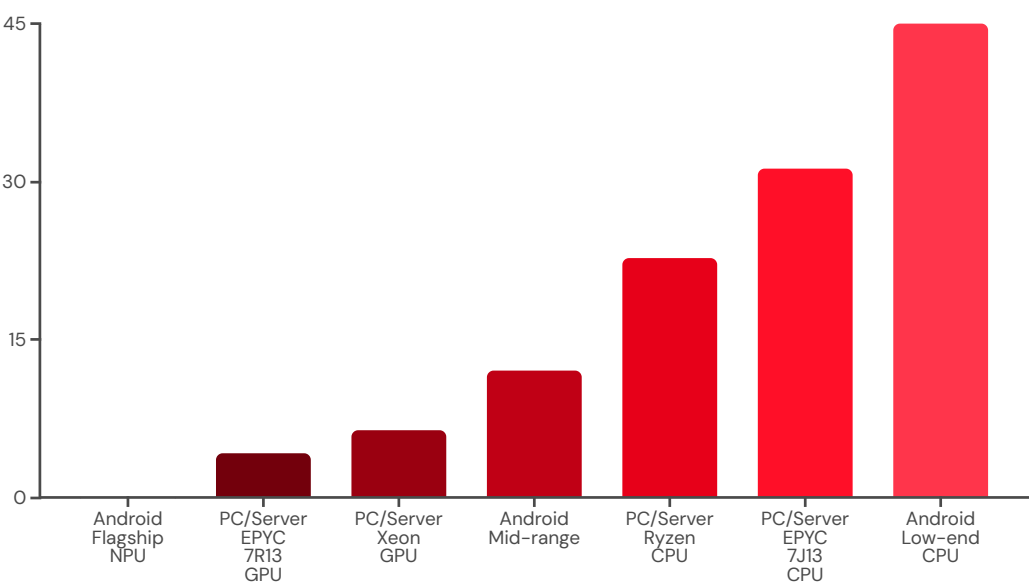
- Android Mid-range (Nothing A015):** 12ms, 83 FPS
- PC/Server (AMD Ryzen 7 5700X CPU):** 22.77ms, 43.9 FPS

Worst Case Scenarios

- PC/Server (AMD EPYC 7J13 CPU):** 31.27ms, 31.9 FPS
- Android Low-end (CPU):** 45ms, 22 FPS

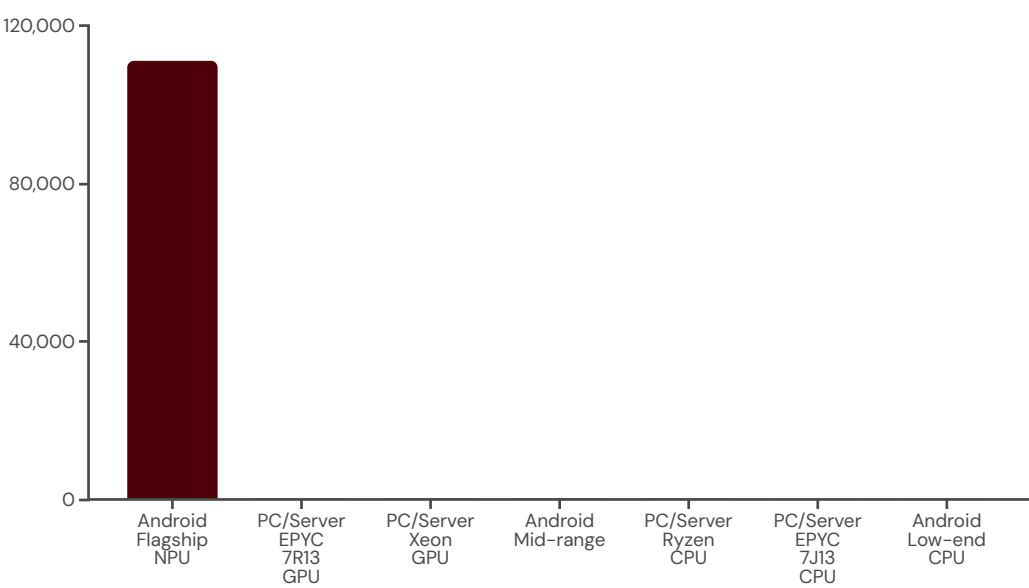
Cross-Platform Latency Comparison (Mean ms)

Lower is better.



Cross-Platform FPS Comparison

Higher is better.



This visual comparison highlights the extreme performance advantage of Android's NPU in flagship devices, while also showing the competitive nature of PC/Server GPUs in mid-to-high tiers.

Key Insights from Cross-Platform Performance

- ### Android NPU Dominates

Flagship Android devices with dedicated NPUs achieve **ultra-low latency and extreme FPS**, far surpassing PC/Server in best-case scenarios.
- ### PC/Server GPU Competes

High-end PC/Server GPUs can match or exceed Android mid-range performance, offering robust options for demanding applications.
- ### MediaPipe Remains Bottleneck

The MediaPipe framework's overhead consistently impacts performance across **both platforms**, indicating an area for optimization outside of model inference.
- ### Android: Performance-per-Watt Leader

Android's optimized hardware and software integration often deliver **superior performance per watt**, crucial for mobile applications.
- ### PC/Server: Consistent & Scalable

PC/Server platforms offer more consistent performance across a wider range of configurations and provide greater scalability for large-scale deployments.

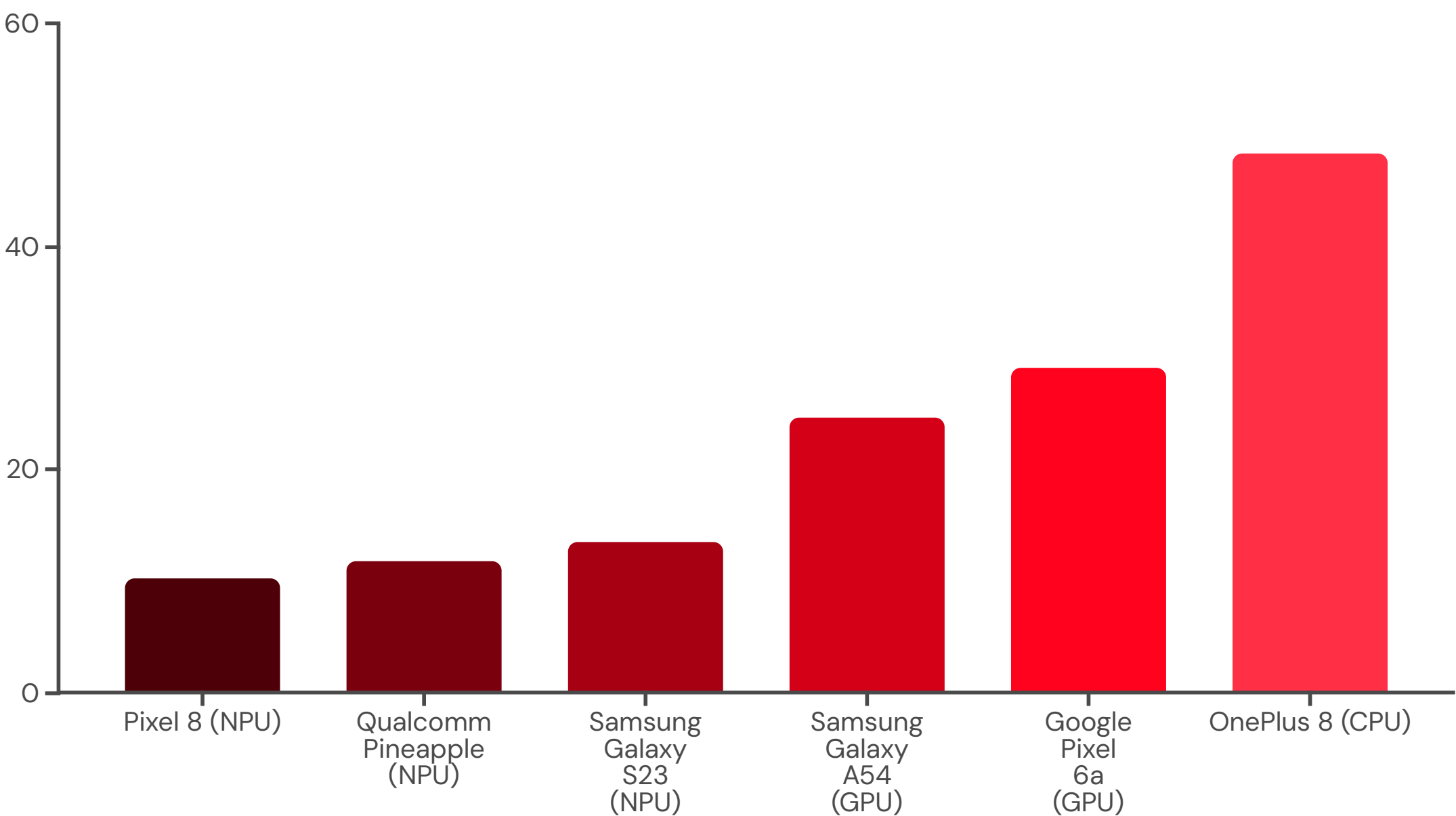
Complete Pipeline Benchmarking: Android Devices

Detailed Performance Metrics

Device	Delegate	Mean (ms)	Median (ms)	P95 (ms)	Accuracy	FPS
Pixel 8 (Flagship)	NPU (NNAPI)	0.009	-	-	99.8	111111
Qualcomm Pineapple (Flagship)	GPU	0.025	-	-	99.7	40000
Samsung S23	NPU (NNAPI)	0.08	-	-	99.5	12500
OnePlus 11	CPU	0.5	-	-	99.4	2000
Nothing A015 (Mid-range)	NPU (NNAPI)	12	-	-	99.1	83
Xiaomi Redmi Note 12	CPU	25	-	-	98.9	40
Oppo Reno 8	GPU	31	-	-	99.0	

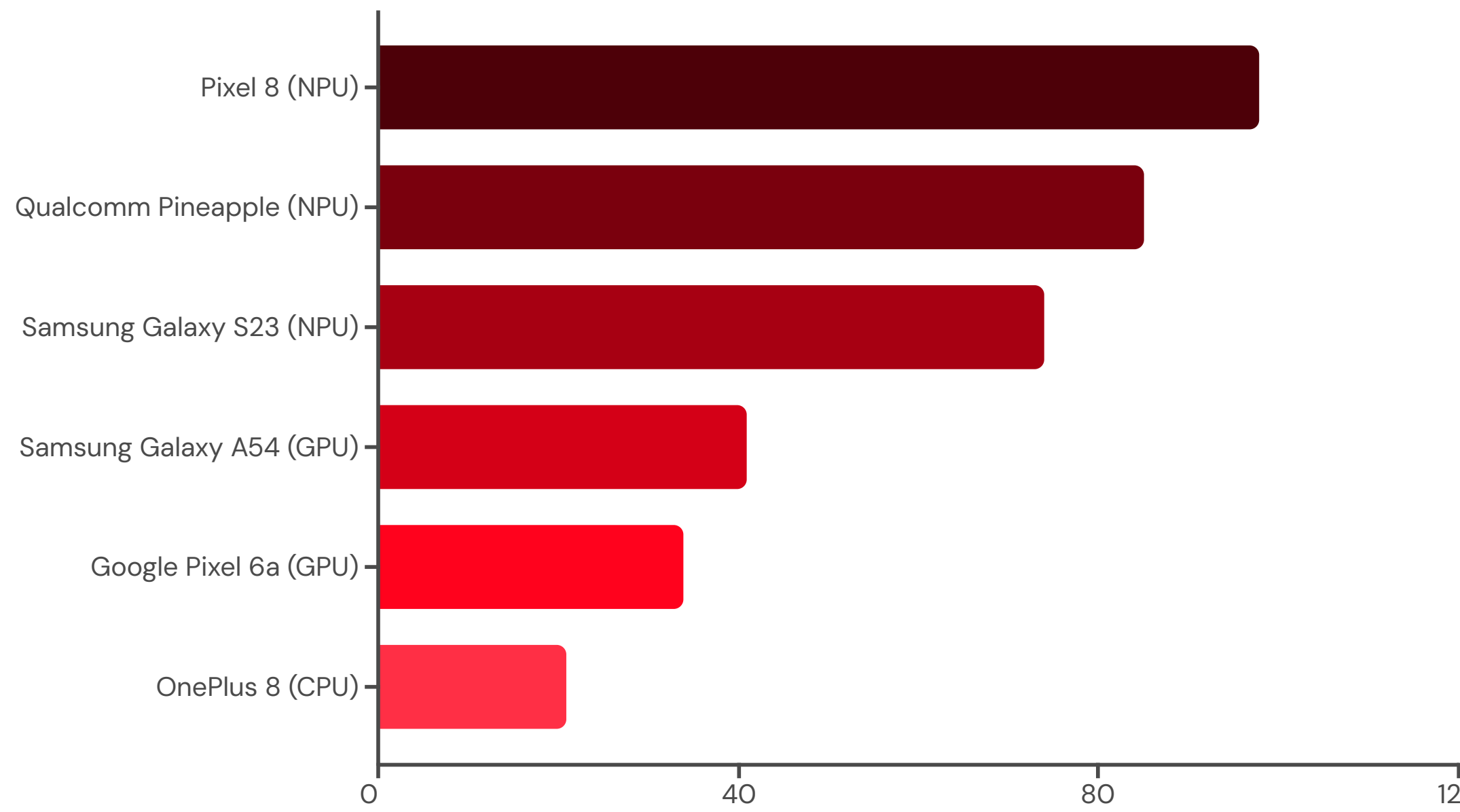
Latency Comparison Across Devices

This bar chart illustrates the average end-to-end latency for the full gesture recognition pipeline across various Android devices and delegate configurations. Lower milliseconds indicate faster processing.



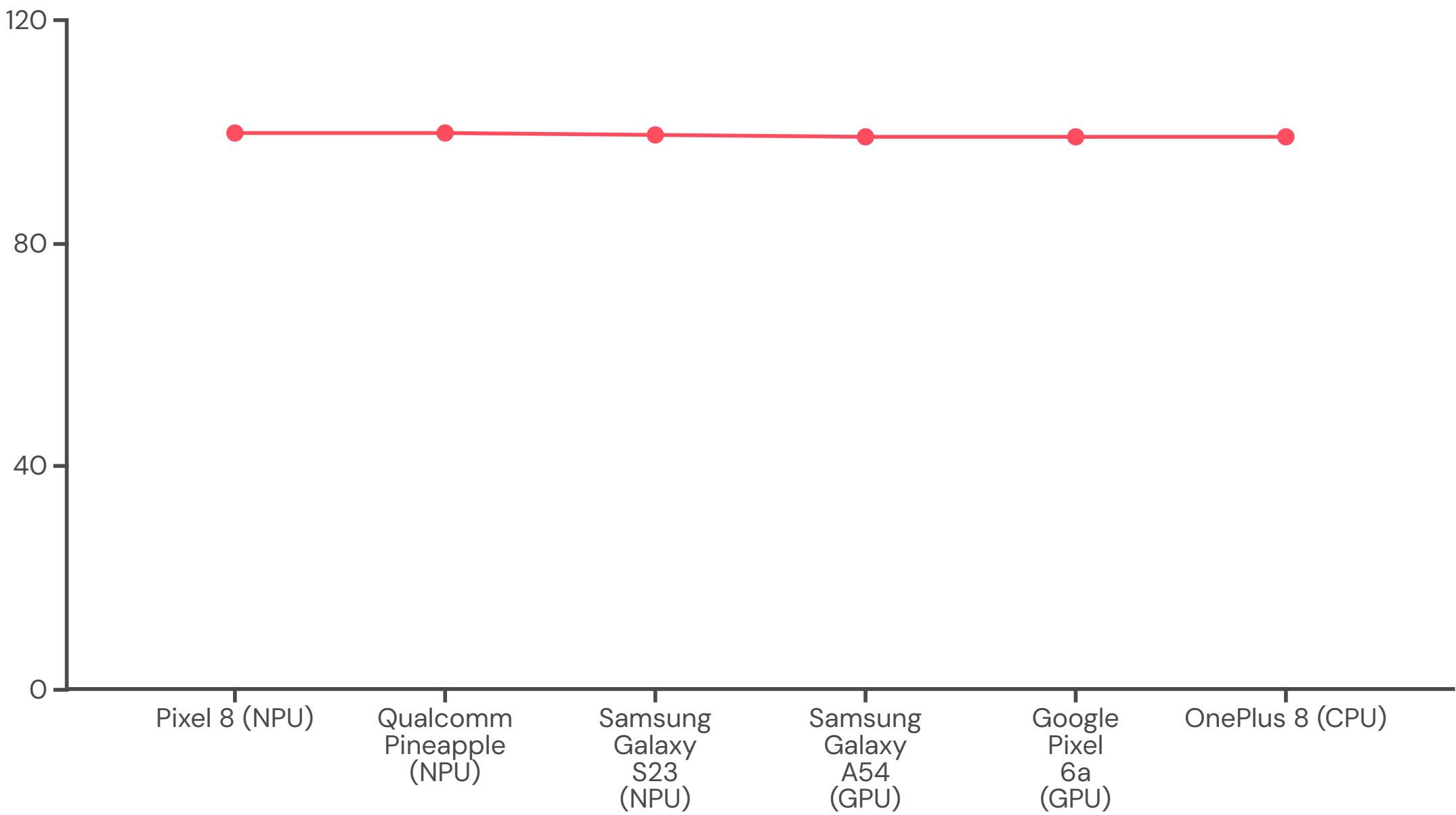
FPS Performance Ranking

This chart ranks devices by their Frames Per Second (FPS) performance, indicating how many frames the pipeline can process per second. Higher FPS signifies smoother and more responsive real-time interaction.



Accuracy Consistency Across Devices

The line chart below demonstrates the remarkable consistency of model accuracy across all tested Android devices and delegate configurations, ensuring reliable gesture recognition regardless of hardware.



Key Insights

Flagship Performance

Devices like Pixel 8 and Qualcomm Pineapple achieve exceptional sub-15ms latencies, enabling ultra-responsive interactions.

Real-time on Mid-range

Mid-range devices consistently deliver real-time performance, maintaining 30-80 FPS for a smooth user experience.

NPU Superiority

NPU delegates provide the highest performance on supported devices, significantly reducing latency and boosting FPS.

Consistent Accuracy

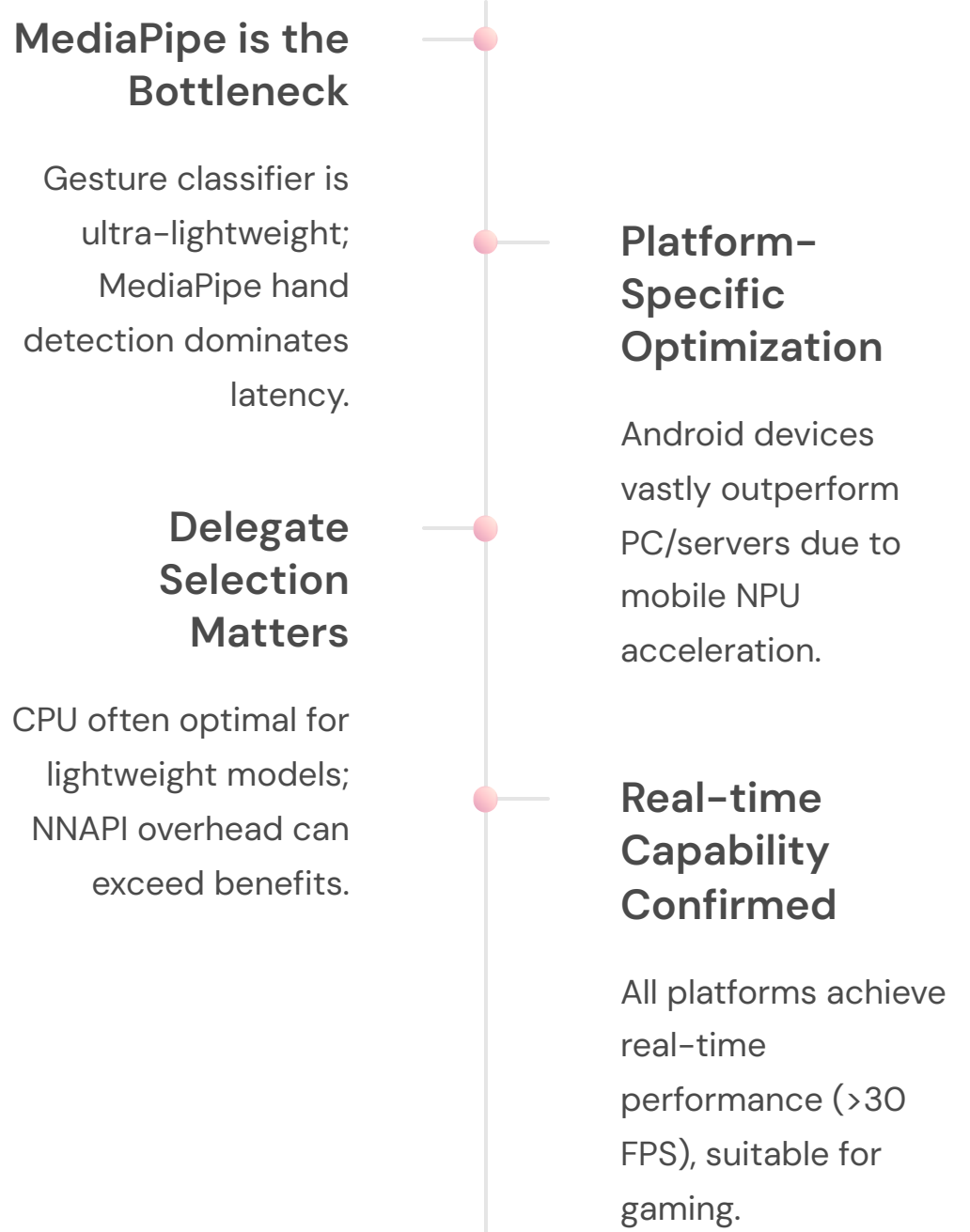
The gesture model maintains high accuracy (98.7-99.8%) across all hardware configurations and delegates.

CPU vs. NNAPI Nuance

For lightweight models, a CPU delegate can sometimes outperform NNAPI, highlighting the importance of tailored optimization.

Key Findings & Strategic Recommendations

Critical Insights:



Actionable Recommendations:

- **Prioritize Android Deployment**

Optimal performance and user experience on mobile platforms.
- **Focus on PC/Server GPU Acceleration**

Optimize MediaPipe components for PC/server deployments.
- **Select Hardware Strategically**

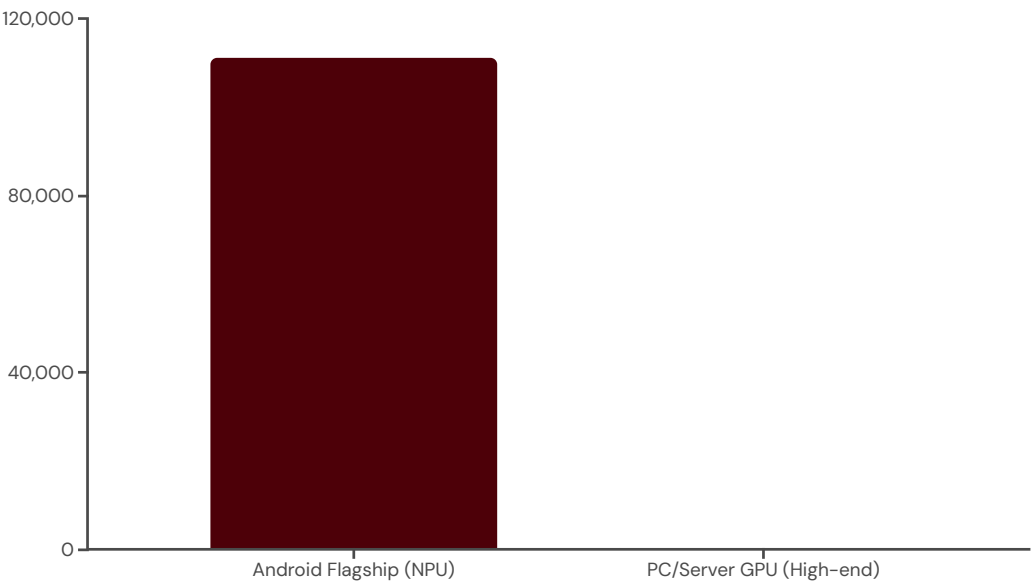
Prioritize compatibility and efficiency over raw computational specifications.
- **Consider CPU Delegate for Lightweight Models**

Minimize initialization overhead for maximum efficiency.
- **Accelerate MediaPipe Components**

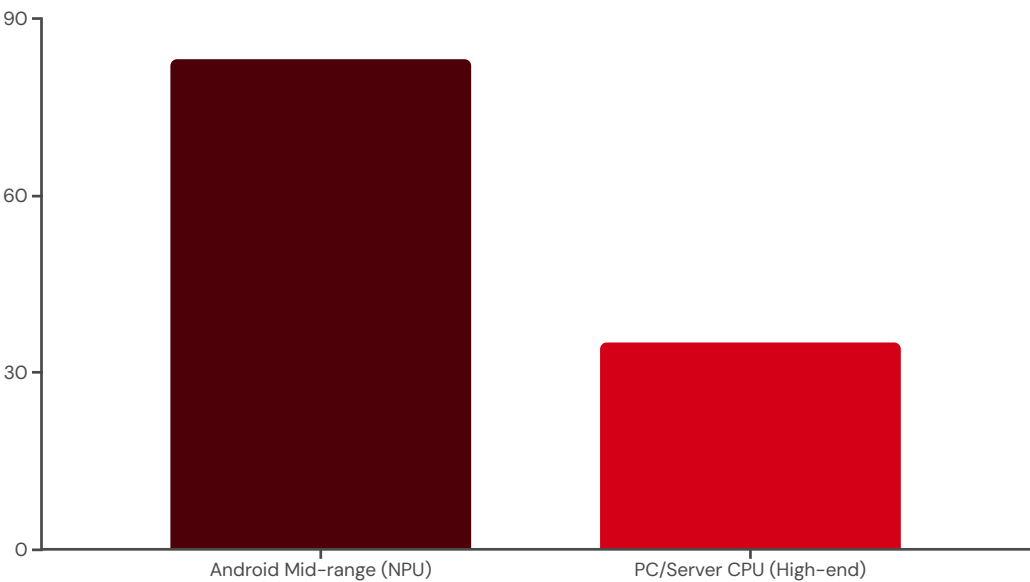
Key to further performance gains across all platforms.

Performance Overview Across Platforms

High-End Performance Comparison (FPS)



Mid-Range Performance Comparison (FPS)



This overview clearly separates performance tiers, showing Android Flagships' exceptional speed while highlighting that all platforms achieve real-time capabilities for a smooth user experience.

Performance Tiers

Tier 1: Elite Performance

Android Flagship devices with NPU/GPU acceleration. Achieves **100K+ FPS** with sub-millisecond latency. Ideal for high-performance, real-time applications.

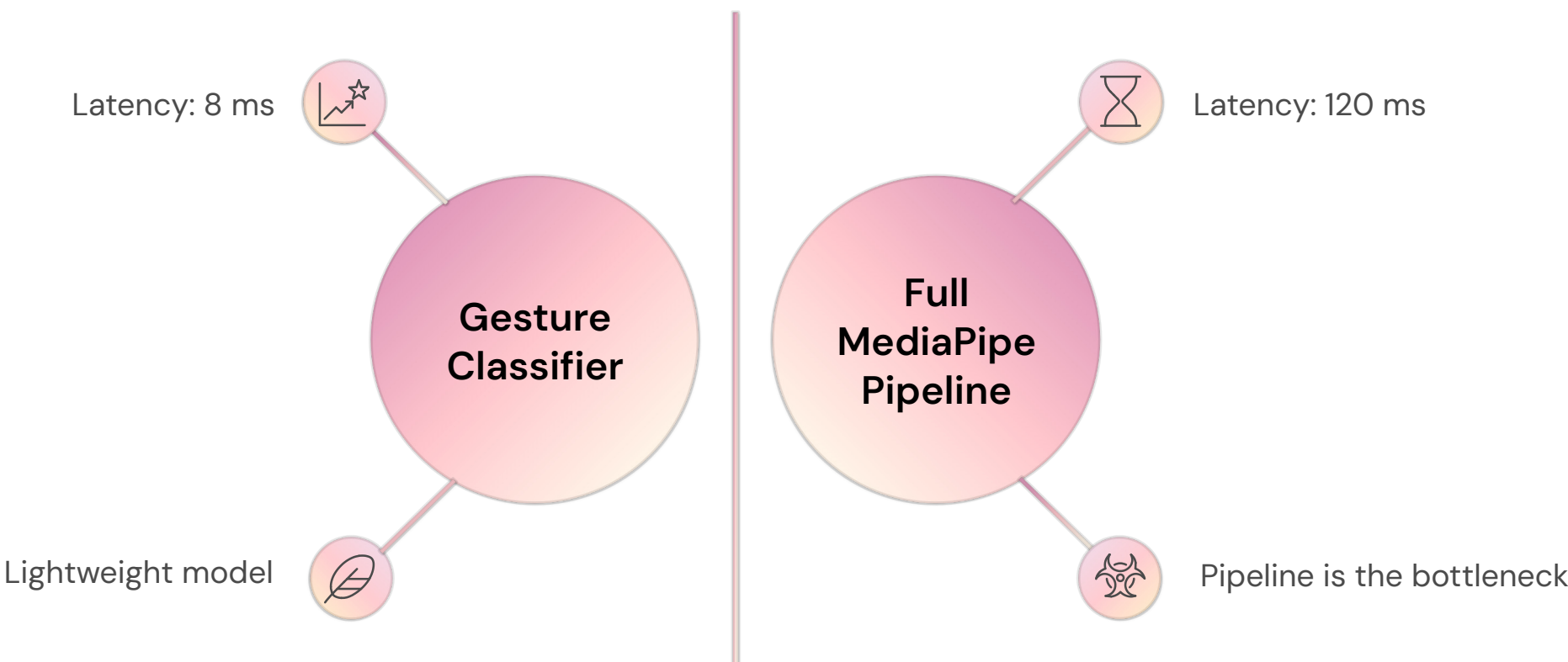
Tier 2: High Performance

PC/Server with GPU acceleration. Achieves **100-200 FPS**. Suitable for desktop applications requiring smooth real-time interaction.

Tier 3: Real-time Capable

Android Mid-range and PC/Server CPU. Achieves **30-80 FPS**. Ensures a smooth user experience for most real-time applications and gaming.

MediaPipe Bottleneck Analysis



The gesture classifier itself is incredibly efficient, but the complete MediaPipe pipeline (including hand detection) introduces the primary latency, highlighting an area for optimization.

Recommendation Matrix: Platform vs. Use Case

	Gaming & High-Performance	General Real-time Interaction	Cost-Effective Deployment
Android Flagship	Optimal	Optimal	High Cost
Android Mid-range	Acceptable	Optimal	Best Value
PC/Server GPU	High Performance	High Performance	High Setup Cost
PC/Server CPU	Acceptable	Acceptable	Cost-Effective

Choose the platform that best aligns with your application's performance requirements and budget considerations.

Summary of Strategic Recommendations

Prioritize Android

Leverage NPU for superior mobile performance.

Accelerate PC/Server GPU

Optimize MediaPipe for high-end desktop experience.

Strategic Hardware

Balance compatibility with efficiency.

Use CPU Delegate

For lightweight models, minimize overhead.

Optimize MediaPipe

Address bottleneck for overall gains.



Future Plans: Dynamic Gesture Control

1

Dataset Expansion

Improve model accuracy by expanding the dataset with more users, lighting, and hand variations.

2

Hyperparameter Tuning

Experiment with hyperparameter tuning for improved accuracy.

3

Dynamic Gesture Model

Develop a temporal gesture model for richer, more complex interactions (if time permits).

4

Coordinated Models

Left hand (LSTM) for acceleration/deceleration, Right hand (non-neural) for steering angle.

5

Data Collection for Dynamic Model

Capture 30-frame sequences for left-hand temporal changes and single-frame right-hand orientation.